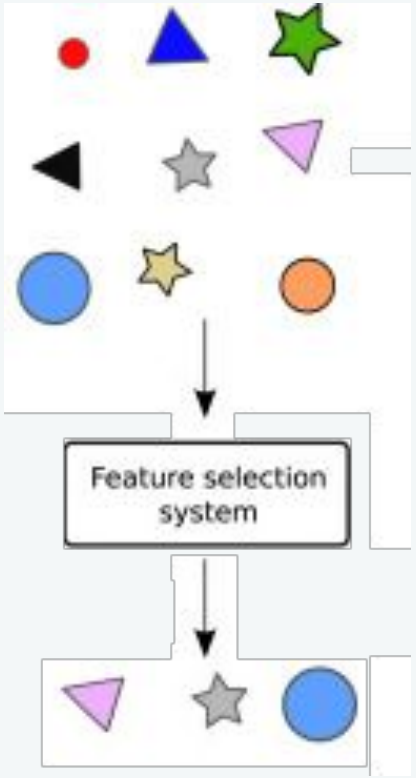
02

Selección de carcterísticas



Selección de características

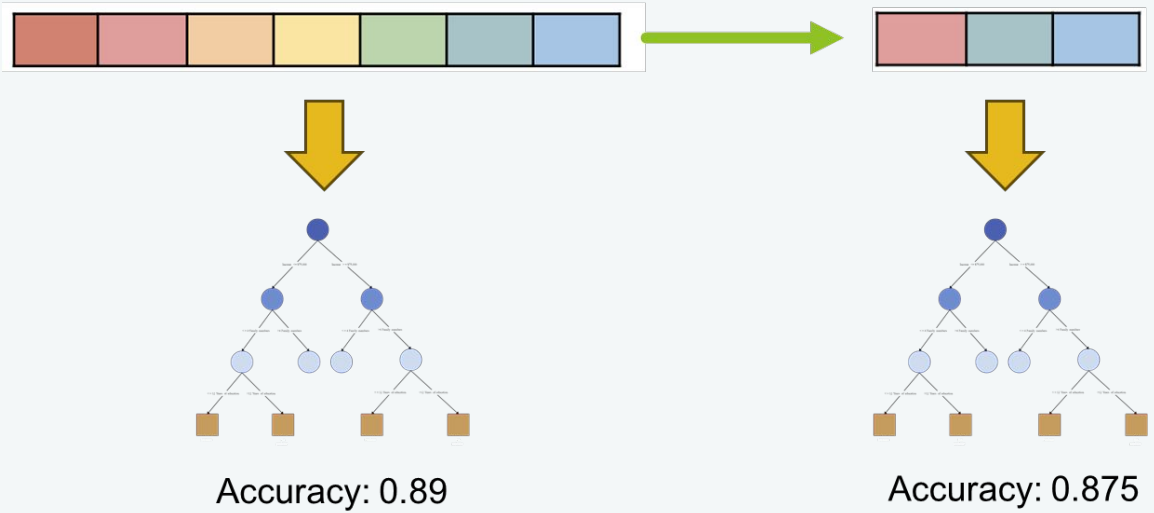
La selección de características (feature selection) es un área dentro del proceso de extracción de conocimiento. Las técnicas dentro de esta área tienen por objetivo identificar las variables más significativas para resolver una determinada tarea.

Ventajas

- Permite reducir la dimensionalidad del dataset.
- Mejora la interpretabilidad del modelo y los resultados.
- Reduce los costos de obtención y almacenamiento de la información.
- Mejora la eficiencia computacional de los algoritmos que hacen uso de la información.
- Ayuda a mitigar el sobreajuste en los algoritmos de machine learning.

Selección de características

El objetivo es encontrar un subconjunto de las variables, atributos o características (columnas) de un dataset, tal que con ese subconjunto se pueda conseguir un modelo de rendimiento similar al que se obtendría utilizando el dataset completo.



Selección de características

Dado un dataset de dimensión N existen 2^N-1 subconjuntos distintos.

Con 30 variables → 1.073.741.823 de combinaciones posibles!!!!!

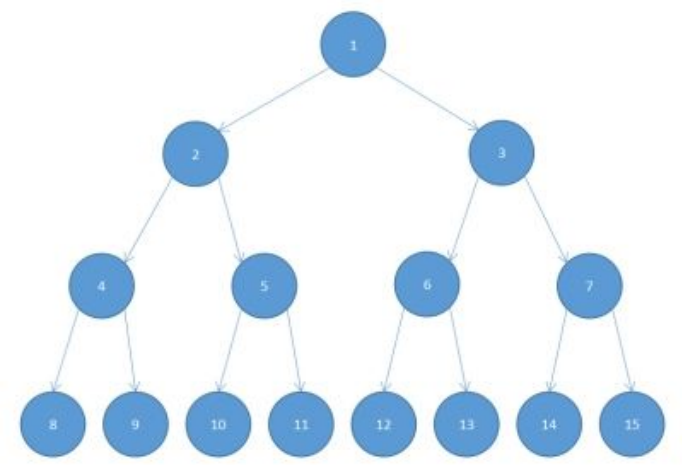
Existen varios enfoques que permiten realizar una selección de características.

- Búsqueda ciega
- Métodos de filtrado
- Métodos empaquetados (wrapper)
- Métodos integrados
- Meta-heurísticas

Búsqueda ciega (Blind search)

La búsqueda ciega consiste en la evaluación del conjunto de soluciones completo. Es decir, dado un conjunto de N características se evalúan las 2^N-1 posibles combinaciones. Se selecciona la mejor de todas.

Con 5 variables → 31 combinaciones



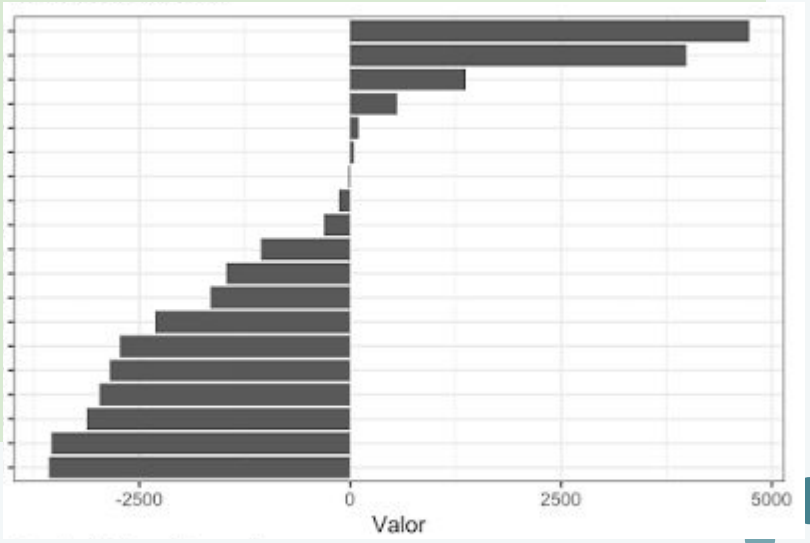
Métodos de filtrado (Filter Methods)

Evalúan la relevancia de cada característica de forma individual sin ningún tipo de sinergia ni combinación entre características.

La relevancia calculada tiende a determinar la correlación de la característica con la variable a predecir.

Las características se con algún índice de de seleccionar aquellas cierto valor.

Es una técnica características rápida del modelo.



Métodos de filtrado (Filter Methods)

Métodos de filtrado más comunes:

- Varianza
- Coeficiente de correlación de Pearson
- Test Chi-cuadrado
- Información mutua
- F-test ANOVA
- Entropía
- Gini
- ...

$$\text{Variance}(X) = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

$$r = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum(x_i-\bar{x})^2\sum(y_i-\bar{y})^2}}$$

$$\chi^2 = \sum\frac{(O-E)^2}{E}$$

$$MI(X,Y) = \sum_{x,y} p(x,y)\log\left(\frac{p(x,y)}{p(x)p(y)}\right)$$

$$F = \frac{\text{Variance Between Groups}}{\text{Variance Within Groups}}$$

Métodos de filtrado

```
from sklearn.feature_selection import VarianceThreshold

variance_selector = VarianceThreshold(threshold=0.1)
X_transform = variance_selector.fit_transform(X)

# La función "corrwith" es de Pandas
correlation_scores = X.corrwith(T)
top_features = correlation_scores.sort_values(ascending=False)

print(top_features.index.tolist())
```

Métodos de filtrado

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.feature_selection import mutual_info_classif
from sklearn.feature_selection import f_classif

chi_selector = SelectKBest(score_func=chi2, k=10)
X_transform = chi_selector.fit_transform(X, T)
print(chi_selector.get_feature_names_out())# Características elegidas

mi_selector = SelectKBest(score_func=mutual_info_classif, k=10)
X_transform = mi_selector.fit_transform(X, T)
print(mi_selector.get_feature_names_out())

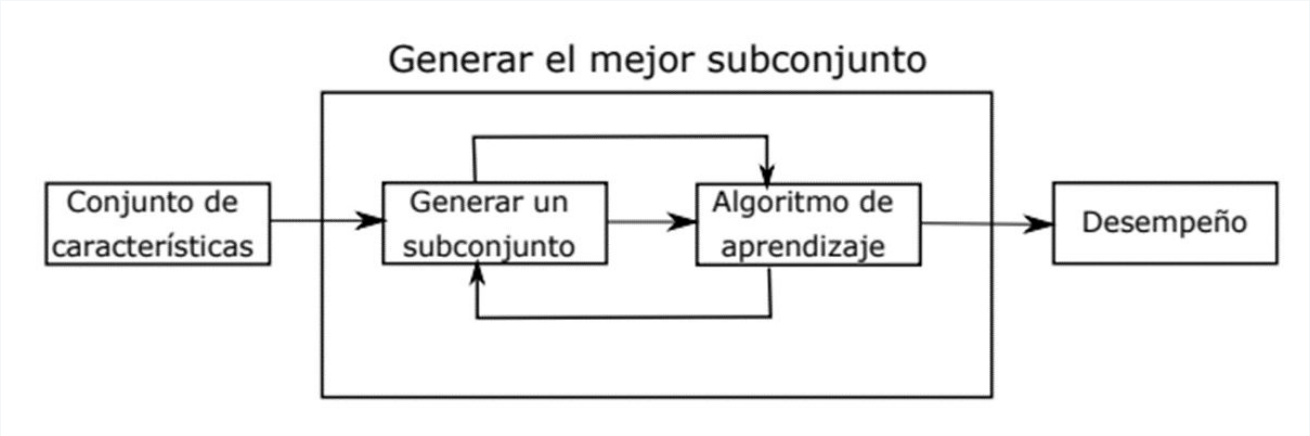
anova_selector = SelectKBest(score_func=f_classif, k=10)
X_transform = anova_selector.fit_transform(X, T)
print(anova_selector.get_feature_names_out())
```

Métodos empaquetados (Wrapper methods)

Son métodos iterativos.

Utiliza un subconjunto de características y se entrena un modelo con el subconjunto seleccionado.

A partir de los resultados obtenidos se agregan o eliminan características del subconjunto.



Métodos empaquetados (Wrapper methods)

Estrategias:

1. Forward selection.
   Comienza con el conjunto vacío, va agregando características, una por una, hasta que la performance del modelo no cambie significativamente.
2. Backward elimination
   Comienza con un conjunto formado por todas las características, mide el modelo, y luego va sacando características mientras la performance no decaiga significativamente.
3. Recursive feature elimination (RFE)
   Parecido al anterior, solo que arma un ranking inicial de variables y comienza a eliminar desde la característica con peor ranking.

Métodos de empaquetado

```
from sklearn.feature_selection import RFE

model_base = RandomForestClassifier()

rfe = RFE(estimator=model_base, n_features_to_select=5)
ranking = rfe.fit(X, T)

print(ranking.get_feature_names_out())
```

Métodos integrados (Embedded methods)

Combinan partes de los métodos de filtrado y de empaquetado. Realizan la selección de las características durante el entrenamiento del modelo.

Son más rápidos que los métodos de empaquetado y a menudo más eficaces que los métodos de filtrado.

Por lo general se implementan como parte del algoritmo de entrenamiento.

Los árboles de decisión los utilizan para elegir la mejor característica al momento de dividir un nodo.

Métodos integrados (Embedded methods)

Regresión LASSO (Least Absolute Shrinkage and Selection Operator), también llamada L1. Es una regresión lineal que intenta hacer que todos sus coeficientes tiendan a cero.

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{n} |w_j|$$

Regresión Ridge (L2). Similar a la anterior, penalizando con más fuerza los coeficientes más grandes.

$$\text{Loss} = \text{MSE} + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$$

Regresión logística regulada. Es como L1, pero realizando una regresión logística para tareas de clasificación.

$$\text{Loss} = -\text{log-likelihood} + \lambda \sum |w_j|$$

L1-SVM. Máquinas de soporte vectorial con regularización L1.

Métodos integrados (Embedded methods)

```
from sklearn.linear_model import Lasso

las = Lasso(alpha=0.1)
model = las.fit(X_train, T_train)
```

Meta heurísticas

Una metaheurística es un algoritmo de alto nivel diseñado para encontrar soluciones buenas o aproximadas a problemas complejos de optimización.

Ejemplos:

- Simples
  - Recocido simulado (simulated annealing)
  - Tabú search
- Poblacionales
  - Algoritmos genéticos
  - Optimización por cúmulo de partículas (PSO, Particle Swarm Optimization)
  - Algoritmo de agujero negro (BHA, Black Hole Algorithm)

Meta heurísticas

Las metaheurísticas trabajan sobre un espacio en R, N dimensional, de posibles soluciones. N = número de características. Cada dimensión (por lo general representada entre 0 y 1, corresponde a una característica.

Evalúan un modelo de clasificación cientos o miles de veces.

Ejemplo.

Una posible solución encontrada por el algoritmo

0.22	0.14	0.78	0.91	0.02	0.41	0.78

Redondeando

0	0	1	1	0	0	1

Las características con “1” son las elegidas.

El resultado encontrado podría no ser el óptimo.

Meta heurísticas

Pseudoalgoritmo de una meta heurística poblacional:

1. Inicializar cada individuo (posible solución) de la población (generación 0)
2. Evaluar los individuos (con una función de aptitud previamente establecida)
3. Iterar hasta converger o hasta alcanzar un máximo de iteraciones
    a. Intercambiar información entre individuos
    b. Obtener la nueva generación
    c. Evaluar los individuos de la nueva generación

El mejor individuo (el que tiene el mejor score según la función de aptitud establecida) es la solución al problema.

Al ser un algoritmo poblacional, se podrían estudiar las soluciones representadas por varios individuos.

Actividades

2. Utilice los métodos de filtrado y empaquetado estudiados para reducir las características que usará un modelo de clasificación que será entrenado con la base de datos “glass aumentada” (alguna resultante de la actividad 1, luego de aplicar alguna técnica de balance de clases).

   Como modelo de clasificación utilice algún modelo de ensemble.

   Determine el valor de accuracy utilizando todos los atributos del dataset. Compare este valor con los obtenidos por los diferentes modelos que se entrenan sobre el dataset reducido (luego de aplicar la selección de características).

   Utilice la técnica de cross-validation para obtener el accuracy de cada modelo.

03

Sistemas de recomendación



Sistemas de recomendación

Un algoritmo de recomendación es un sistema de toma de decisiones o de puntuación diseñado para sugerir elementos, contenido o acciones relevantes a los usuarios en función de sus preferencias, comportamientos y datos contextuales.

- Filtrado basado en contenido: Sugiere elementos similares a los ya consumidos, basándose en las características del propio contenido
- Filtrado colaborativo: Recomienda elementos analizando los patrones de usuarios con gustos parecidos (el principio de "usuarios que llevaron esto también les gustó aquello").

Pueden basarse en reglas, en similitudes o utilizar aprendizaje automático.

El objetivo principal de los algoritmos de recomendación es mejorar la experiencia del usuario y la toma de decisiones al proporcionar sugerencias personalizadas que reduzcan la sobrecarga de información.